

1.数组和链表基础

数组和链表的基本原理和实现

数组（顺序存储）基本原理

暂且可以把「数组」分为两大类，一类是「静态数组」，一类是「动态数组」。

「静态数组」就是一块连续的内存空间，我们可以通过索引来访问这块内存空间中的元素，这才是数组的原始形态。

而「动态数组」是编程语言为了方便我们使用，在静态数组的基础上帮我们添加了一些常用的 API，比如 `push`, `insert`, `remove` 等等方法，这些 API 可以让我们更方便地操作数组元素，不用自己去写代码实现这些操作。

静态数组本质上就是一块**连续的**内存空间，`int arr[10]` 这个语句我们可以得知：

- 1、我们知道这块内存空间的首地址（数组名 `arr` 就指向这块内存空间的首地址）。
- 2、我们知道了每个元素的类型（比如 `int`），也就是知道了每个元素占用的内存空间大小（比如一个 `int` 占 4 字节，32 bit）。
- 3、这块内存空间是连续的，其大小为 `10 * sizeof(int)` 即 40 字节。
- 4、因为数组的名字 `arr` 就指向整块内存的首地址，所以数组名 `arr` 就是一个指针。你直接取这个地址的值，就是第一个元素的值。也就是说，`*arr` 的值就是 `arr[0]`，即第一个元素的值。

所以，我们获得了数组的超能力「随机访问」：只要给定任何一个数组索引，我可以在 $O(1)$ 的时间内直接获取到对应元素的值。

因为我可以通过首地址和索引直接计算出目标元素的内存地址。计算机的内存寻址时间可以认为是 $O(1)$ ，所以数组的随机访问时间复杂度是 $O(1)$ 。

- 增：
 - 在末尾追加元素： $O(1)$ 。
 - 在中间（非末尾）插入元素： $O(N)$ 。
- 删：
 - 删除末尾元素： $O(1)$ 。
 - 删除中间（非末尾）元素： $O(N)$ 。
- 查：给定指定索引，查询索引对应的元素的值，时间复杂度 $O(1)$ 。
- 改：给定指定索引，修改索引对应的元素的值，时间复杂度 $O(1)$ 。

动态数组代码实现

关键点一、自动扩缩容

关键点二、索引越界的检查

关键点三、删除元素谨防内存泄漏

```
#include <iostream>
#include <stdexcept>
#include <vector>

template<typename E>
class MyArrayList {
private:
    // 真正存储数据的底层数组
    E* data;
    // 记录当前元素个数
    int size;

    // 最大元素容量
    int cap;

    // 默认初始容量
    static const int INIT_CAP = 1;

public:
    MyArrayList() {
        this->data = new E[INIT_CAP];
        this->size = 0;
        this->cap = INIT_CAP;
    }

    MyArrayList(int initCapacity) {
        this->data = new E[initCapacity];
        this->size = 0;
        this->cap = initCapacity;
    }

    // 增
    void addLast(E e) {
        // 看 data 数组容量够不够
        if (size == cap) {
            resize(2 * cap);
        }
        // 在尾部插入元素
        data[size] = e;
        size++;
    }

    void add(int index, E e) {
        // 检查索引越界
        checkPositionIndex(index);
```

```

        // 看 data 数组容量够不够
        if (size == cap) {
            resize(2 * cap);
        }

        // 搬移数据 data[index..] -> data[index+1..]
        // 给新元素腾出位置
        for (int i = size - 1; i >= index; i--) {
            data[i + 1] = data[i];
        }

        // 插入新元素
        data[index] = e;

        size++;
    }

    void addFirst(E e) {
        add(0, e);
    }

    // 删
    E removeLast() {
        if (size == 0) {
            throw std::out_of_range("NoSuchElementException");
        }
        // 可以缩容, 节约空间
        if (size == cap / 4) {
            resize(cap / 2);
        }

        E deletedVal = data[size - 1];
        // 删除最后一个元素
        // 必须给最后一个元素置为 null, 否则会内存泄漏
        data[size - 1] = E();
        size--;

        return deletedVal;
    }

    E remove(int index) {
        // 检查索引越界
        checkElementIndex(index);

        // 可以缩容, 节约空间
        if (size == cap / 4) {
            resize(cap / 2);
        }
    }

```

```

        E deletedVal = data[index];

        // 搬移数据 data[index+1..] -> data[index..]
        for (int i = index + 1; i < size; i++) {
            data[i - 1] = data[i];
        }

        data[size - 1] = E();
        size--;

        return deletedVal;
    }

    E removeFirst() {
        return remove(0);
    }

    // 查
    E get(int index) {
        // 检查索引越界
        checkElementIndex(index);

        return data[index];
    }

    // 改
    E set(int index, E element) {
        // 检查索引越界
        checkElementIndex(index);
        // 修改数据
        E oldVal = data[index];
        data[index] = element;
        return oldVal;
    }

    // 工具方法
    int getSize() {
        return size;
    }

    bool isEmpty() {
        return size == 0;
    }

    // 将 data 的容量改为 newCap
    void resize(int newCap) {
        E* temp = new E[newCap];

        for (int i = 0; i < size; i++) {
            temp[i] = data[i];

```

```

    }

    // 释放原数组内存
    delete[] data;

    data = temp;
    cap = newCap;
}

bool isElementIndex(int index) {
    return index >= 0 && index < size;
}

bool isPositionIndex(int index) {
    return index >= 0 && index <= size;
}

// 检查 index 索引位置是否可以存在元素
void checkElementIndex(int index) {
    if (!isElementIndex(index)) {
        throw std::out_of_range("Index out of bounds");
    }
}

// 检查 index 索引位置是否可以添加元素
void checkPositionIndex(int index) {
    if (!isPositionIndex(index)) {
        throw std::out_of_range("Index out of bounds");
    }
}

void display() {
    std::cout << "size = " << size << " cap = " << cap << std::endl;
    for (int i = 0; i < size; i++) {
        std::cout << data[i] << " ";
    }
    std::cout << std::endl;
}

~MyArrayList() {
    delete[] data;
}

};

int main() {
    // 初始容量设置为 3
    MyArrayList<int> arr(3);

    // 添加 5 个元素

```

```

    for (int i = 1; i <= 5; i++) {
        arr.addLast(i);
    }

    arr.remove(3);
    arr.add(1, 9);
    arr.addFirst(100);
    int val = arr.removeLast();

    // 100 1 9 2 3
    for (int i = 0; i < arr.getSize(); i++) {
        std::cout << arr.get(i) << std::endl;
    }

    return 0;
}

```

链表（链式存储）基本原理

一条链表并不需要一整块连续的内存空间存储元素。链表的元素可以分散在内存空间的天涯海角，通过每个节点上的 `next`, `prev` 指针，将零散的内存块串联起来形成一个链式结构。

这样做的好处很明显，首先就是可以提高内存的利用效率，链表的节点不需要挨在一起，给点内存 `new` 出来一个节点就能用，操作系统会觉得这娃好养活。

**单链表

```

class ListNode {
public:
    int val;
    ListNode *next;
    ListNode(int x) : val(x), next(NULL) {}
};

// 输入一个数组，转换为一条单链表
ListNode* createLinkedList(std::vector<int> arr) {
    if (arr.empty()) {
        return nullptr;
    }
    ListNode* head = new ListNode(arr[0]);
    ListNode* cur = head;
    for (int i = 1; i < arr.size(); i++) {
        cur->next = new ListNode(arr[i]);
        cur = cur->next;
    }
    return head;
}

```

查/改

单链表的遍历/查找/修改

比方说，我想访问单链表的每一个节点，并打印其值，可以这样写：

```
// 创建一条单链表
ListNode* head = createLinkedList({1, 2, 3, 4, 5});

// 遍历单链表
for (ListNode* p = head; p != nullptr; p = p->next) {
    std::cout << p->val << std::endl;
}
```

类似的，如果是要通过索引访问或修改链表中的某个节点，也只能用 for 循环从头结点开始往后找，直到找到索引对应的节点，然后进行访问或修改。

增

在单链表头部插入新元素

我们会持有单链表的头结点，所以只需要将插入的节点接到头结点之前，并将新插入的节点作为头结点即可。

```
// 创建一条单链表
ListNode* head = createLinkedList({1, 2, 3, 4, 5});

// 在单链表头部插入一个新节点 0
ListNode* newNode = new ListNode(0);
newNode->next = head;
head = newNode;

// 现在链表变成了 0 -> 1 -> 2 -> 3 -> 4 -> 5
```

在单链表尾部插入新元素

直接看代码吧，很简单：

```
// 创建一条单链表
ListNode* head = createLinkedList({1, 2, 3, 4, 5});

// 在单链表尾部插入一个新节点 6
ListNode* p = head;
// 先走到链表的最后一个节点
while (p->next != nullptr) {
    p = p->next;
}
// 现在 p 就是链表的最后一个节点
```

```
// 在 p 后面插入新节点
p->next = new ListNode(6);

// 现在链表变成了 1 -> 2 -> 3 -> 4 -> 5 -> 6
```

当然，如果我们持有对链表尾节点的引用，那么在尾部插入新节点的操作就会变得非常简单，不用每次从头去遍历了。这个优化会在后面具体实现双链表时介绍。

在单链表中间插入新元素

这个操作稍微有点复杂，我们还是要先找到要插入位置的前驱节点，然后操作前驱节点把新节点插入进去：

```
// 创建一条单链表
ListNode* head = createLinkedList({1, 2, 3, 4, 5});

// 在第 3 个节点后面插入一个新节点 66
// 先要找到前驱节点，即第 3 个节点
ListNode* p = head;
for (int i = 0; i < 2; i++) {
    p = p->next;
}
// 此时 p 指向第 3 个节点
// 组装新节点的后驱指针
ListNode* newNode = new ListNode(66);
newNode->next = p->next;

// 插入新节点
p->next = newNode;

// 现在链表变成了 1 -> 2 -> 3 -> 66 -> 4 -> 5
```

删

在单链表中删除一个节点

删除一个节点，首先要找到要被删除节点的前驱节点，然后把这个前驱节点的 next 指针指向被删除节点的下一个节点。这样就能把被删除节点从链表中摘除了。

```
// 创建一条单链表
ListNode* head = createLinkedList({1, 2, 3, 4, 5});

// 删除第 4 个节点，要操作前驱节点
ListNode* p = head;
for (int i = 0; i < 2; i++) {
    p = p->next;
}

// 此时 p 指向第 3 个节点，即要删除节点的前驱节点
```



```
// 把第 4 个节点从链表中摘除
p->next = p->next->next;

// 现在链表变成了 1 -> 2 -> 3 -> 5
```

在单链表尾部删除元素

这个操作比较简单，找到倒数第二个节点，然后把它的 `next` 指针置为 `null` 就行了：

```
// 创建一条单链表
ListNode* head = createLinkedList({1, 2, 3, 4, 5});

// 删除尾节点
ListNode* p = head;
// 找到倒数第二个节点
while (p->next->next != nullptr) {
    p = p->next;
}

// 此时 p 指向倒数第二个节点
// 把尾节点从链表中摘除
p->next = nullptr;

// 现在链表变成了 1 -> 2 -> 3 -> 4
```

基于虚拟头尾节点的链表代码实现

关键点

关键点一、同时持有头尾节点的引用

对于单链表来说，持有尾部节点的引用也有优化效果。比如你要在单链表尾部添加元素，如果没有尾部节点的引用，你就需要遍历整个链表找到尾部节点，时间复杂度是 $O(n)$ ；如果有尾部节点的引用，就可以在 $O(1)$ 的时间复杂度内完成尾部添加元素的操作。删除单链表尾结点的时候，是不是也得遍历到倒数第二个节点（尾结点的前驱），才能通过指针操作把尾结点删掉？那么这个时候，你不就可以顺便把尾结点的引用给更新了吗？

关键点二、虚拟头尾节点

它的原理很简单，就是在创建双链表时就创建一个虚拟头节点和一个虚拟尾节点，无论双链表是否为空，这两个节点都存在。这样就不会出现空指针的问题，可以避免很多边界情况的处理。

举例来说，假设虚拟头尾节点分别是 `dummyHead` 和 `dummyTail`，那么一条空的双链表长这样：

```
dummyHead <-> dummyTail
```

如果你添加 1, 2, 3 几个元素，那么链表长这样：

```
dummyHead <-> 1 <-> 2 <-> 3 <-> dummyTail
```

你以前要把在头部插入元素、在尾部插入元素和在中间插入元素几种情况分开讨论，现在有了头尾虚拟节点，无论链表是否为空，都只需要考虑在中间插入元素的情况就可以了，这样代码会简洁很多。

当然，虚拟头结点会多占用一点内存空间，但是比起给你解决的麻烦，这点空间消耗是划算的。

对于单链表，虚拟头结点有一定的简化作用，但虚拟尾节点没有太大作用。

虚拟节点是内部实现，对外不可见

虚拟节点是你内部实现数据结构的技巧，对外是不可见的。比如按照索引获取元素的 `get(index)` 方法，都是从真实节点开始计算索引，而不是从虚拟节点开始计算。

关键点三、内存泄露

```
// 假设单链表头结点 head = 1 -> 2 -> 3 -> 4 -> 5

// 删除单链表头结点
head = head.next;

// 此时 head = 2 -> 3 -> 4 -> 5
```

细心的读者可能认为这样写会有内存泄露的问题，因为原来的那个头结点 1 的 next 指针没有断开，依然指向着节点 2。

但实际上这样写是 OK 的，因为 Java 的垃圾回收的判断机制是看这个对象是否被别人引用，而并不会 care 这个对象是否还引用着别人。

那个节点 1 的 next 指针确实还指向着节点 2，但是并没有别的指针引用节点 1 了，所以节点 1 最终会被垃圾回收器回收释放。所以说这个场景和数组中删除元素的场景是不一样的，你可以再仔细思考一下。

不过呢，删除节点时，最好还是把被删除节点的指针都置为 null，这是个好习惯，不会有什么代价，还可能避免一些潜在的问题。所以在下面的实现中，无论是否有必要，我都会把被删除节点上的指针置为 null。

单链表代码实现

```
#include <iostream>
#include <stdexcept>

template <typename E>
class MyLinkedList2 {
private:
```

```

// 节点结构
struct Node {
    E val;
    Node* next;

    Node(E value) : val(value), next(nullptr) {}
};

Node* head;
// 实际的尾部节点引用
Node* tail;
int size_;

public:
    MyLinkedList2() {
        head = new Node(E());
        tail = head;
        size_ = 0;
    }

    ~MyLinkedList2() {
        Node* current = head;
        while (current != nullptr) {
            Node* next = current->next;
            delete current;
            current = next;
        }
    }

    void addFirst(E e) {
        Node* newNode = new Node(e);
        newNode->next = head->next;
        head->next = newNode;
        if (size_ == 0) {
            tail = newNode;
        }
        size_++;
    }

    void addLast(E e) {
        Node* newNode = new Node(e);
        tail->next = newNode;
        tail = newNode;
        size_++;
    }

    void add(int index, E element) {
        checkPositionIndex(index);

        if (index == size_) {

```

```

        addLast(element);
        return;
    }

    Node* prev = head;
    for (int i = 0; i < index; i++) {
        prev = prev->next;
    }
    Node* newNode = new Node(element);
    newNode->next = prev->next;
    prev->next = newNode;
    size_++;
}

E removeFirst() {
    if (isEmpty()) {
        throw std::out_of_range("No elements to remove");
    }
    Node* first = head->next;
    head->next = first->next;
    if (size_ == 1) {
        tail = head;
    }
    size_--;
    E val = first->val;
    delete first;
    return val;
}

E removeLast() {
    if (isEmpty()) {
        throw std::out_of_range("No elements to remove");
    }

    Node* prev = head;
    while (prev->next != tail) {
        prev = prev->next;
    }
    E val = tail->val;
    delete tail;
    prev->next = nullptr;
    tail = prev;
    size_--;
    return val;
}

E remove(int index) {
    checkElementIndex(index);

    Node* prev = head;

```

```

        for (int i = 0; i < index; i++) {
            prev = prev->next;
        }

        Node* nodeToRemove = prev->next;
        prev->next = nodeToRemove->next;
        // 删除的是最后一个元素
        if (index == size_ - 1) {
            tail = prev;
        }
        size_--;
        E val = nodeToRemove->val;
        delete nodeToRemove;
        return val;
    }

    // ***** 查 *****

    E getFirst() {
        if (isEmpty()) {
            throw std::out_of_range("No elements in the list");
        }
        return head->next->val;
    }

    E getLast() {
        if (isEmpty()) {
            throw std::out_of_range("No elements in the list");
        }
        return tail->val;
    }

    E get(int index) {
        checkElementIndex(index);
        Node* p = getNode(index);
        return p->val;
    }

    // ***** 改 *****

    E set(int index, E element) {
        checkElementIndex(index);
        Node* p = getNode(index);

        E oldVal = p->val;
        p->val = element;

        return oldVal;
    }

```

```

// ***** 其他工具函数 *****
int size() {
    return size_;
}

bool isEmpty() {
    return size_ == 0;
}

private:
    bool isElementIndex(int index) {
        return index >= 0 && index < size_;
    }

    bool isPositionIndex(int index) {
        return index >= 0 && index <= size_;
    }

    // 检查 index 索引位置是否可以存在元素
    void checkElementIndex(int index) {
        if (!isElementIndex(index)) {
            throw std::out_of_range("Index: " + std::to_string(index) + ",
size_: " + std::to_string(size_));
        }
    }

    // 检查 index 索引位置是否可以添加元素
    void checkPositionIndex(int index) {
        if (!isPositionIndex(index)) {
            throw std::out_of_range("Index: " + std::to_string(index) + ",
size_: " + std::to_string(size_));
        }
    }

    // 返回 index 对应的 Node
    // 注意: 请保证传入的 index 是合法的
    Node* getNode(int index) {
        Node* p = head->next;
        for (int i = 0; i < index; i++) {
            p = p->next;
        }
        return p;
    }
};

int main() {
    MyLinkedList2<int> list;
    list.addFirst(1);
    list.addFirst(2);
    list.addLast(3);
}

```

```

list.addLast(4);
list.add(2, 5);

std::cout << list.removeFirst() << std::endl; // 2
std::cout << list.removeLast() << std::endl; // 4
std::cout << list.remove(1) << std::endl; // 5

std::cout << list.getFirst() << std::endl; // 1
std::cout << list.getLast() << std::endl; // 3
std::cout << list.get(1) << std::endl; // 3

return 0;
}

```

双链表

```

#include <iostream>
#include <stdexcept>

template<typename E>
class MyLinkedList {
    // 虚拟头尾节点
    struct Node {
        E val;
        Node* next;
        Node* prev;

        Node(E value) : val(value), next(nullptr), prev(nullptr) {}
    };

    Node* head;
    Node* tail;
    int size;

public:
    // 构造函数初始化虚拟头尾节点
    MyLinkedList() {
        head = new Node(E());
        tail = new Node(E());
        head->next = tail;
        tail->prev = head;
        size = 0;
    }

    ~MyLinkedList() {
        while (size > 0) {
            removeFirst();
        }
        delete head;
    }
}

```

```

        delete tail;
    }

    // ***** 增 *****

void addLast(E e) {
    Node* x = new Node(e);
    Node* temp = tail->prev;

    temp->next = x;
    x->prev = temp;
    // temp <-> x

    x->next = tail;
    tail->prev = x;
    // temp <-> x <-> tail
    size++;
}

void addFirst(E e) {
    Node* x = new Node(e);
    Node* temp = head->next;
    // head <-> temp
    temp->prev = x;
    x->next = temp;

    head->next = x;
    x->prev = head;
    // head <-> x <-> temp
    size++;
}

void add(int index, E element) {
    checkPositionIndex(index);
    if (index == size) {
        addLast(element);
        return;
    }

    // 找到 index 对应的 Node
    Node* p = getNode(index);
    Node* temp = p->prev;
    // temp <-> p

    // 新要插入的 Node
    Node* x = new Node(element);

    p->prev = x;
    temp->next = x;

```



```

        x->prev = temp;
        x->next = p;

        // temp <-> x <-> p

        size++;
    }

    // ***** 删 *****

E removeFirst() {
    if (size < 1) {
        throw std::out_of_range("No elements to remove");
    }
    // 虚拟节点的存在是我们不用考虑空指针的问题
    Node* x = head->next;
    Node* temp = x->next;
    // head <-> x <-> temp
    head->next = temp;
    temp->prev = head;

    E val = x->val;
    delete x;
    // head <-> temp

    size--;
    return val;
}

E removeLast() {
    if (size < 1) {
        throw std::out_of_range("No elements to remove");
    }
    Node* x = tail->prev;
    Node* temp = tail->prev->prev;
    // temp <-> x <-> tail

    tail->prev = temp;
    temp->next = tail;

    E val = x->val;
    x->prev = nullptr;
    x->next = nullptr;
    delete x;
    // temp <-> tail

    size--;
    return val;
}

```

```

E remove(int index) {
    checkElementIndex(index);
    // 找到 index 对应的 Node
    Node* x = getNode(index);
    Node* prev = x->prev;
    Node* next = x->next;
    // prev <-> x <-> next
    prev->next = next;
    next->prev = prev;

    E val = x->val;
    x->prev = nullptr;
    x->next = nullptr;
    delete x;
    // prev <-> next

    size--;
    return val;
}

// ***** 查 *****

E get(int index) {
    checkElementIndex(index);
    // 找到 index 对应的 Node
    Node* p = getNode(index);

    return p->val;
}

E getFirst() {
    if (size < 1) {
        throw std::out_of_range("No elements in the list");
    }

    return head->next->val;
}

E getLast() {
    if (size < 1) {
        throw std::out_of_range("No elements in the list");
    }

    return tail->prev->val;
}

// ***** 改 *****

E set(int index, E val) {
    checkElementIndex(index);

```

```

        // 找到 index 对应的 Node
        Node* p = getNode(index);

        E oldVal = p->val;
        p->val = val;

        return oldVal;
    }

    // ***** 其他工具函数 *****

    int getSize() const {
        return size;
    }

    bool isEmpty() const {
        return size == 0;
    }

    void display() {
        std::cout << "size = " << size << std::endl;
        for (Node* p = head->next; p != tail; p = p->next) {
            std::cout << p->val << " <-> ";
        }
        std::cout << "nullptr" << std::endl;
        std::cout << std::endl;
    }

private:
    Node* getNode(int index) {
        checkElementIndex(index);
        Node* p = head->next;
        // TODO: 可以优化, 通过 index 判断从 head 还是 tail 开始遍历
        for (int i = 0; i < index; i++) {
            p = p->next;
        }
        return p;
    }

    bool isElementIndex(int index) const {
        return index >= 0 && index < size;
    }

    bool isPositionIndex(int index) const {
        return index >= 0 && index <= size;
    }

    // 检查 index 索引位置是否可以存在元素
    void checkElementIndex(int index) const {
        if (!isElementIndex(index))

```

```

        throw std::out_of_range("Index: " + std::to_string(index) + ",
Size: " + std::to_string(size));
    }

    // 检查 index 索引位置是否可以添加元素
    void checkPositionIndex(int index) const {
        if (!isPositionIndex(index))
            throw std::out_of_range("Index: " + std::to_string(index) + ",
Size: " + std::to_string(size));
    }
};

```

循环双向链表

```

struct node

{

    int data;
    node *right;
    node *left;

};

// 循环双向链表的操作类

class CircularDoublyLinkedList
{
private:
    node *head;
    int count;

public:

    CircularDoublyLinkedList()
    {
        head = new node;
        head->right = head;
        head->left = head;
        head->data = 0;
        count = 0;
    }

    void append(int value)
    {

```

```

    node *newNode = new node;
    newNode->data = value;
    node *last = head->left; // 当前尾节点

    newNode->right = head;
    newNode->left = last;
    last->right = newNode;
    head->left = newNode;
    count++;

}

int getLength() const
{
    return count;
}

bool insertNode(int pos, int value)
{
    if (pos < 1 || pos > count + 1)
    {
        cout << "错误: 插入位置 " << pos << " 无效。" << endl;
        return false;
    }
    node *current = head;
    for (int i = 0; i < pos - 1; i++)
    {
        current = current->right;
    }
    node *newNode = new node;
    newNode->data = value;

    newNode->right = current->right;
    newNode->left = current;
    current->right->left = newNode;
    current->right = newNode;
    count++;
    return true;
}

bool deleteNode(int pos)
{
    if (count == 0 || pos < 1 || pos > count)
    {
        cout << "错误: 删除位置 " << pos << " 无效或链表为空。" << endl;
        return false;
    }
    node *prev = head;
    for (int i = 0; i < pos - 1; ++i)
    {
        prev = prev->right;
    }

```

```

        node *toDelete = prev->right;
        node *next = toDelete->right;
        prev->right = next;
        next->left = prev;
        delete toDelete;
        count--;
        return true;
    }
void printForward() const
{
    if (count == 0)
    {
        cout << "链表为空。" << endl;
        return;
    }
    cout << "正向输出 (Head -> ... -> Last): ";
    node *current = head->right; // 从第一个数据结点开始
    while (current != head)
    { // 循环直到回到头结点
        cout << current->data;
        if (current->right != head)
        {
            cout << " -> ";
        }
        current = current->right;
    }
    cout << endl;
}
void printBackward() const
{
    if (count == 0)
    {
        cout << "链表为空。" << endl;
        return;
    }
    cout << "反向输出 (Last -> ... -> Head): ";
    node *current = head->left; // 从最后一个数据结点开始
    while (current != head)
    { // 循环直到回到头结点
        cout << current->data;
        if (current->left != head)
        {
            cout << " -> ";
        }
        current = current->left;
    }
    cout << endl;
}

```

~CircularDoublyLinkedList()

```

    {
        // 重复删除第一个数据结点直到链表为空
        while (count > 0)
        {
            deleteNode(1);
        }
        // 最后删除头结点
        delete head;
        head = nullptr;
    }
};

```

数组链表的转换

环形数组技巧及实现

环形数组技巧利用求模（余数）运算，将普通数组变成逻辑上的环形数组，可以让我们用 $O(1)$ 的时间在数组头部增删元素。

环形数组的关键在于，它维护了两个指针 `start` 和 `end`，`start` 指向第一个有效元素的索引，`end` 指向最后一个有效元素的下一个位置索引。

这样，当我们在数组头部添加或删除元素时，只需要移动 `start` 索引，而在数组尾部添加或删除元素时，只需要移动 `end` 索引。

当 `start`, `end` 移动超出数组边界（ < 0 或 $\geq \text{arr.length}$ ）时，我们可以通过求模运算 `%` 让它们转一圈到数组头部或尾部继续工作，这样就实现了环形数组的效果。

在我的代码中，环形数组的区间被定义为左闭右开的，即 `[start, end)` 区间包含数组元素。所以其他的方法都是以左闭右开区间为基础实现的。

理论上，你可以随意设计区间的开闭，但一般设计为左闭右开区间是最方便处理的。

因为这样初始化 `start = end = 0` 时，区间 `[0, 0)` 中没有元素，但只要让 `end` 向右移动（扩大）一位，区间 `[0, 1)` 就包含一个元素 `0` 了。

如果你设置为两端都开的区间，那么让 `end` 向右移动一位后开区间 `(0, 1)` 仍然没有元素；如果你设置为两端都闭的区间，那么初始区间 `[0, 0]` 就已经包含了一个元素。这两种情况都会给边界处理带来不必要的麻烦，如果你非要使用的话，需要在代码中做一些特殊处理。

```

#include <iostream>
#include <stdexcept>
#include <vector>
#include <ostream>

template<typename T>
class CycleArray {
    std::vector<T> arr;
    int start;
    int end;

```

```

int count;

// 自动扩缩容辅助函数
void resize(int newSize) {
    // 创建新的数组并复制元素
    std::vector<T> newArr(newSize);
    for (int i = 0; i < count; ++i) {
        newArr[i] = arr[(start + i) % arr.size()];
    }
    arr = std::move(newArr);
    // 重置 start 和 end 指针
    start = 0;
    end = count;
}

public:
    CycleArray() : CycleArray(1) {}

    explicit CycleArray(int size)
        : arr(size), start(0), end(0), count(0) {}

    // 在数组头部添加元素，时间复杂度 O(1)
    void addFirst(const T &val) {
        // 当数组满时，扩容为原来的两倍
        if (isFull()) {
            resize(arr.size() * 2);
        }
        // 因为 start 是闭区间，所以先左移，再赋值
        start = (start - 1 + arr.size()) % arr.size();
        arr[start] = val;
        count++;
    }

    // 删除数组头部元素，时间复杂度 O(1)
    void removeFirst() {
        if (isEmpty()) {
            throw std::runtime_error("Array is empty");
        }
        // 因为 start 是闭区间，所以先赋值，再右移
        arr[start] = T();
        start = (start + 1) % arr.size();
        count--;
        // 如果数组元素数量减少到原大小的四分之一，则减小数组大小为一半
        if (count > 0 && count == arr.size() / 4) {
            resize(arr.size() / 2);
        }
    }

    // 在数组尾部添加元素，时间复杂度 O(1)
    void addLast(const T &val) {

```



```

        if (isFull()) {
            resize(arr.size() * 2);
        }
        // 因为 end 是开区间，所以是先赋值，再右移
        arr[end] = val;
        end = (end + 1) % arr.size();
        count++;
    }

    // 删除数组尾部元素，时间复杂度 O(1)
    void removeLast() {
        if (isEmpty()) {
            throw std::runtime_error("Array is empty");
        }
        // 因为 end 是开区间，所以先左移，再赋值
        end = (end - 1 + arr.size()) % arr.size();
        arr[end] = T();
        count--;
        // 扩容
        if (count > 0 && count == arr.size() / 4) {
            resize(arr.size() / 2);
        }
    }

    // 获取数组头部元素，时间复杂度 O(1)
    T getFirst() const {
        if (isEmpty()) {
            throw std::runtime_error("Array is empty");
        }
        return arr[start];
    }

    // 获取数组尾部元素，时间复杂度 O(1)
    T getLast() const {
        if (isEmpty()) {
            throw std::runtime_error("Array is empty");
        }
        // end 是开区间，指向的是下一个元素的位置，所以要减 1
        return arr[(end - 1 + arr.size()) % arr.size()];
    }

    bool isFull() const {
        return count == arr.size();
    }

    int size() const {
        return count;
    }

    bool isEmpty() const {

```

```

        return count == 0;
    }
};

```

我们都说，在数组增删头部元素的时间复杂度是 $O(N)$ ，因为需要搬移元素。但是，如果我们使用环形数组，其实是可以在 $O(1)$ 的时间复杂度内增删头部元素的。

环形数组也可以获取指定索引的元素（随机访问），只不过不是直接访问对应索引，而是要通过 `start` 计算出真实索引，但计算和访问的时间复杂度依然是 $O(1)$ ；

环形数组也可以在指定索引插入元素，当然也要做数据搬移，和普通数组一样，复杂度是 $O(N)$ 。

标准库中提供的动态数组容器底层并没有用环形数组技巧的原因：如果用环形数组，增删查改的所有操作都会涉及 $\%$ 求模运算，这个操作是比较消耗性能的。尤其像数组的 `get` 方法，调用频率会非常非常高，如果每次调用都多一步 $\%$ 运算，加起来的性能损耗远大于环形数组带来的收益，因为数组很少在头部增删元素。如果你非要在头部增删，应该使用更合适的其他数据结构。

所以一般只会在双端队列这种场景下使用环形数组，标准的动态数组并没有使用这个技巧。不是不能用，而是算总账不划算。

跳表核心原理

利用**空间换时间**的思想，用额外的空间记录额外的信息，增删查改的时间复杂度都能优化到 $O(\log N)$ 。

一条普通的单链表长这样：

```

index  0  1  2  3  4  5  6  7  8  9
node   a->b->c->d->e->f->g->h->i->j

```

如果我们想查询索引为 7 的元素是什么，只能从索引 0 头结点开始往后遍历，直到遍历到索引 7，找到目标节点 `h`。

而跳表则是这样的：

```

indexLevel  0-----8-----10
indexLevel  0-----4-----8-----10
indexLevel  0-----2-----4-----6-----8-----10
indexLevel  0--1--2--3--4--5--6--7--8--9--10
nodeLevel   a->b->c->d->e->f->g->h->i->j->k

```

跳表相当于在原链表的基础上，增加了多层索引，每向上一层，索引节点的数量减少一半，索引的间隔变为 2 倍，所以索引的高度是 $\log N$ ， N 代表链表中元素的个数。

此时，如果我们想查询索引为 7 的元素，可以从最高层索引开始一层一层地往下找：

首先最高层的第一个索引区间是 `[0, 8]`，可以确定索引 7 在这个区间内，所以从下一层的节点 0 开始搜索；

第二层从节点 0 开始，索引区间 `[0, 4]` 不包含索引 7，继续往右移动到节点 4，索引区间 `[4, 8]` 包含索引 7，所以从下一层的节点 4 开始搜索；

第三层从节点 4 开始，索引区间 `[4, 6]` 不包含索引 7，继续往右移动到节点 6，索引区间 `[6, 8]` 包含索引 7，所以从下一层的节点 6 开始搜索；

第四层从节点 6 开始，索引区间 `[6, 7]` 包含索引 7，最终找到目标节点 `h`。

这个搜索过程中，会经过 $\log N$ 层索引，在每层索引中移动的次数不会超过 2 次（因为上层索引区间在下一层被分为两半），所以跳表的查询时间复杂度是 $O(\log N)$ 。

1、上面的例子只展示了查询操作，但跳表肯定得支持插入和删除操作，这就涉及到索引层中节点的动态调整，你需要保证每一层的索引区间尽可能二分，这样才能保证索引层的高度为 $\log N \log N$ ，否则时间复杂度就会退化。

2、不仅仅是查找索引对应的节点，跳表还可以运用到更通用的场景，比如说有序键值对的存储和查找。实际上，跳表的使用场景和后面我们会学习到的二叉搜索树非常类似，只不过跳表的代码实现相较于自平衡二叉搜索树要简单很多。

稀疏矩阵

三元组表是存储稀疏矩阵最直观、最简单的一种方法。它的核心思想是：**放弃存储大量的零元素，只记录那些非零元素的信息。**

具体来说，它通过记录每个非零元素的三个关键信息来描述整个矩阵：

- **行号 (Row)**：该非零元素所在的行。
- **列号 (Column)**：该非零元素所在的列。
- **值 (Value)**：该非零元素自身的数值。

这三个信息构成一个“三元组”，形式为 `(row, col, value)`。整个稀疏矩阵就可以表示为这些三元组的集合。

一个完整的三元组表数据结构通常包含两部分：

1. 矩阵的元数据：

- 矩阵的总行数 (`rows`)。
- 矩阵的总列数 (`cols`)。
- 非零元素的总个数 (`nonZeroNum`)。

2. 三元组数组 (或列表)：

- 一个用来存放所有非零元素三元组的数组。数组的长度就是 `nonZeroNum`。

为了方便处理，这个三元组数组通常会按照某种顺序（例如，按行优先，或按列优先）进行排序。

优点:

- **结构简单直观**: 非常容易理解和编程实现。
- **节省存储空间**: 对于非常稀疏的矩阵, 空间利用率极高。
- **易于创建**: 从稠密矩阵转换或直接构建都非常方便。

缺点:

- **随机访问效率低**: 如果你想获取任意位置 (i, j) 的值, 无法像二维数组那样通过索引直接定位。你必须遍历整个三元组列表来查找是否存在 行= i 且 列= j 的项。最坏情况下, 时间复杂度为 $O(N)$, 其中 N 是非零元素的个数。
- **矩阵运算复杂**: 进行矩阵加法、乘法等运算时, 需要对两个矩阵的三元组列表进行匹配和遍历, 算法相对复杂且效率不高。例如, 矩阵转置操作虽然简单 (只需交换每个三元组的行和列号并重新排序), 但其他运算就比较麻烦。

总结

三元组表是学习稀疏矩阵的入门级和基础性存储结构。它完美地体现了用**时间换空间**的思想, 通过牺牲随机访问的效率, 极大地优化了存储。在实际应用中, 对于需要频繁进行矩阵运算的场景, 通常会采用更高效的**压缩稀疏行 (CSR)** 或 **压缩稀疏列 (CSC)** 等格式。

1. 为什么需要快速转置?

对于一个以三元组表存储的稀疏矩阵 M , 其转置矩阵 MT 的三元组表可以通过将 M 中每个三元组 (r, c, v) 变为 (c, r, v) 来得到。

- **普通转置法 (Simple Transpose)**:
 - 遍历 M 的三元组表, 将每个三元组的行号和列号互换。
 - **问题**: 简单互换后得到的 MT 三元组表是**无序**的 (它依然是按 M 的行序排列, 但 MT 需要按列序排列)。如果需要进行后续运算, 这个无序的表通常需要额外进行一次复杂的**排序**操作。排序的计算复杂度通常较高。
- **快速转置法 (Fast Transpose)**:
 - 避免对整个三元组表进行排序, 而是通过巧妙的计数和定位, **一次性**将元素放置到 MT 三元组表的**正确位置**。

2. 快速转置算法的核心思想

快速转置算法主要利用了空间换时间的思想, 通过两个辅助数组来实现 $O(Cols + NonZeroNum)$ 的时间复杂度 (比排序的 $O(N \log N)$ 更快, 其中 N 是非零元素个数)。

核心步骤:

步骤 1: 统计每列非零元素的个数 (使用 `num` 数组)

1. 创建一个长度为 `Cols` 的数组 `num`, 用于统计 M 中每一列 (即 MT 中每一行) 有多少个非零元素。
2. 初始化 `num` 数组所有元素为 0。

3. 遍历 M 的三元组表，对于每个三元组 (r,c,v)，执行 `num[c]++`。

作用： 得到 MT 中每一行将有多少个元素。

步骤 2: 确定每行第一个非零元素的位置 (使用 `cpot` 数组)

1. 创建一个长度为 `Cols` 的数组 `cpot` (Column Position)，用于记录 MT 的三元组表中**每一行第一个元素**应该存放的起始位置。
2. MT 的第一个元素始终位于索引 0，所以： `cpot[0] = 0`。
3. 从 `i=1` 开始到 `Cols-1`，计算 `cpot[i]`：

`cpot[i]=cpot[i-1]+num[i-1]`

即，第 `i` 行的起始位置等于前一行 `i-1` 的起始位置加上前一行 `i-1` 的非零元素个数。

作用： 快速定位 M 中某一列 `c` 的元素在 MT 数组中的存放位置。

步骤 3: 循环遍历并放置元素

1. 创建一个新的三元组表 MT 用于存放转置结果。
2. 再次遍历 M 的三元组表，对于每个三元组 (r,c,v):
 - 目标行号 `r'` 就是 `c`。
 - 目标列号 `c'` 就是 `r`。
 - 通过 `cpot[c]` 确定 (r',c',v) 应该放置在 MT 三元组表中的**位置 p**。
 - 将 (c,r,v) 存入 MT 的位置 `p`。
 - **更新定位指针：** 执行 `cpot[c]++`，确保下一个属于列 `c` 的元素会被放在紧邻的下一个位置。

```
class SparseMatrix;
class Trituple {
    friend class SparseMatrix;
private:
    int row, col;
    int data;
};

class SparseMatrix {
public:
    SparseMatrix(int MaxRow, int MaxCol);
    SparseMatrix Mat_Fast_Transpose();
private:
    int Rows, Cols, NonZero_Terms;
    vector<Trituple> SArray;
};

SparseMatrix SparseMatrix::Mat_Fast_Transpose() {
    SparseMatrix b(Cols, Rows);
```

```

int i, j;
int *x = new int[Cols];
int *y = new int[Cols];
b.NonZero_Terms = NonZero_Terms;
if (NonZero_Terms > 0) {
    b.SMArray.resize(NonZero_Terms);
    for (int i = 0; i < Cols; i++) x[i] = 0;
    for (int i = 0; i < NonZero_Terms; i++) x[SMArray[i].col]++;
    y[0] = 0;
    for (int i = 1; i < Cols; i++) y[i] = y[i - 1] + x[i - 1];
    for (int i = 0; i < NonZero_Terms; i++) {
        j = y[SMArray[i].col];
        b.SMArray[j].row = SMArray[i].col;
        b.SMArray[j].col = SMArray[i].row;
        b.SMArray[j].data = SMArray[i].data;
        y[SMArray[i].col]++;
    }
}
delete[] x;
delete[] y;
return b;
}'''

```